

DEVELOPMENT_PLAN_PLAIN

Development Plan — In Plain English

Plain-language companion to `docs/planning/DEVELOPMENT_PLAN.md`. For the exact spec, read that / its PDF.

In one sentence

This is the build plan: the milestones M0–M12 broken into concrete work packages, each with what to build, how to test it, and the exit gate it must pass before you move on.

Why this matters to you

When you sit down for a session, this doc tells you exactly where you are and what's next. You open the milestone tracker (§12), find the lowest milestone that isn't done, open its work-package table, build it in safe order, pass its gate, mark it done. It's designed so a future session (or a future you) can resume without re-figuring out the plan.

The big ideas, simply

- **Work packages (WP).** Each milestone is broken into one or more work packages like WP-3.1, each with a clear definition of done and a test.
- **Governed-first.** Build the data spine and the metric registry before any agents. No code ever hand-writes SQL or computes stats (that's grounding rules G1–G5).
- **TDD on SQL.** Write the test before the model. The keystones — sessionization, funnel monotonicity, revenue reconciliation, `decompose_change` — get golden tests because they fail silently (wrong numbers, not errors).
- **Three maturity levels.** **L1** (M0–M7) = the governed spine works end to end. **L2** (M8–M10) = the full 7-agent loop, benchmarked at $\geq 85\%$ vs $\leq 45\%$ baseline. **L3** (M11) = autonomy and depth.

- **Three parallel tracks.** After M0, you can build the data spine, the data-independent math (`stats-mcp`), and the memory/finance pieces at the same time.

What you actually do (in order)

1. **Each session:** run `/resume` , pick the next unfinished work package, plan it against `DEPENDENCY_MAP.md` , build it test-first, pass the exit gate, mark it done, update `CLAUDE.md` §10.
2. **M0 (WP-0.1):** repo scaffold + GCP/IAM/ADC + dbt config. Gate: `dbt debug` connects, ADC authenticates.
3. **M1–M2:** macros, source, seed, then staging models. Gate: macros compile, staging tests pass.
4. **M3 (WP-3.1, WP-3.2):** the data keystone — `int_ga4__sessionized` then `int_ga4__funnel_steps` . Gate: `session_key` unique, funnel monotonicity golden test passes.
5. **M4–M5:** the marts (`fct_funnel` , `fct_daily_funnel` , `fct_orders` , dims), then the registry compiles live. Gate: revenue reconciles to the cent, channels = 10, 0 dangling refs.
6. **M6 / M6b:** the grounding MCP pair (`semantic-mcp` + `warehouse-mcp`) and the stats server with `decompose_change` . Gate: query round-trips, `decompose_change` golden test passes.
7. **M7:** the minimal agent loop (L1 done). Gate: one anomaly → brief in <5 min, 0 hallucinated columns.
8. **M8–M11:** memory, the full agent loop, the eval harness + CI, then autonomy. These are the L2 and L3 levels.

Easy things to get wrong

- **Starting a milestone before its deps are green.** The tracker order enforces this — respect it.
- **Skipping golden tests on keystones.** They fail silently; a wrong keystone poisons every number downstream.
- **Letting the registry drift from the marts.** A column rename must update exactly one YAML; CI referential-integrity catches drift.
- **Falling behind on continuity docs.** When you add or rename an artifact, update `DEPENDENCY_MAP.md` and this tracker; canonical-name changes also update the Bible and `CLAUDE.md` .
- **Treating targets as measurements.** The 85% / <5 min / 0-hallucination numbers are goals until the eval actually runs at M10.

Glossary — the exact words, demystified

- **Work package (WP-x.y)** — one unit of buildable work inside a milestone.
- **Exit gate** — the test/condition that must pass before the milestone counts as done.
- **DoD** — definition of done; the concrete checklist for a work package.
- **L1 / L2 / L3** — maturity levels: Intern MVP, Strong portfolio, Top-1% undergrad.
- **Keystone** — a load-bearing artifact (sessionization, the registry, `decompose_change`) that breaks everything if wrong.
- **Golden test** — checks output against a hand-worked correct value.
- **TDD** — test-driven development: write the test first, then make it pass.
- **/resume** — a custom slash command that reads the tracker and reports the next unfinished package.
- **Artifact IDs (A0.1, A3.1, ...)** — stable references to specific files/components, used across all planning docs.

When to open the real doc

Open `DEVELOPMENT_PLAN.md` (and its copy under `pdf/`) at the start of each session for the \$12 tracker, and when you need a work package's exact files, spec doc, workflow, and definition of done (\$7–\$9). It is your living resume point.